

3^η Εργαστηριακή Άσκηση

Γρηγόρης Δελατόλας (03119202)

Γεώργιος Κυριάκος (03119204)

Άσκηση 1

Κώδικας:

```
/*
 * simplesync.c
 *
 * A simple synchronization exercise.
 *
 * Vangelis Koukis <vkoukis@cslab.ece.ntua.gr>
 * Operating Systems course, ECE, NTUA
 */

#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>

/*
 * POSIX thread functions do not return error numbers in errno,
 * but in the actual return value of the function call instead.
 * This macro helps with error reporting in this case.
 */
#define perror_thread(ret, msg) \
    do { errno = ret; perror(msg); } while (0)

#define N 10000000

/* Dots indicate lines where you are free to insert code at will */
/* ... */

#if defined(SYNC_ATOMIC) ^ defined(SYNC_MUTEX) == 0
# error You must #define exactly one of SYNC_ATOMIC or SYNC_MUTEX.
#endif

#if defined(SYNC_ATOMIC)
# define USE_ATOMIC_OPS 1
#else
# define USE_ATOMIC_OPS 0
#endif

pthread_mutex_t mutex;
int ptr = 0;
int x = 0;
```

```

void *increase_fn(void *arg)
{
    int i;
    volatile int *ip = arg;

    fprintf(stderr, "About to increase variable %d times\n", N);
    for (i = 0; i < N; i++) {
        if (USE_ATOMIC_OPS) {
            /* ... */

            /* You can modify the following line */
            //++(*ip);
            x = __sync_add_and_fetch (&ptr, 1);

            /* ... */
        }
        else {
            /* ... */
            if (0 != (errno = pthread_mutex_lock(&mutex)))
            {
                perror("pthread_mutex_lock failed");
                exit(EXIT_FAILURE);
            }

            /* You cannot modify the following line */

            ++(*ip);

            if (0 != (errno = pthread_mutex_unlock(&mutex))) {
                perror("pthread_mutex_unlock failed");
                exit(EXIT_FAILURE);
            }
            /* ... */
        }
    }
    fprintf(stderr, "Done increasing variable.\n");

    return NULL;
}

```

```

void *decrease_fn(void *arg)
{
    int i;
    volatile int *ip = arg;

    fprintf(stderr, "About to decrease variable %d times\n", N);
    for (i = 0; i < N; i++) {

        if (USE_ATOMIC_OPS) {
            /* ... */
            /* You can modify the following line */
            x = __sync_sub_and_fetch (&ptr, -1);

            /* ... */
        }
        else {
            /* ... */
            if (0 != (errno = pthread_mutex_lock(&mutex)))
            {
                perror("pthread_mutex_lock failed");
                exit(EXIT_FAILURE);
            }

            /* You cannot modify the following line */

            --(*ip);

            if (0 != (errno = pthread_mutex_unlock(&mutex))) {

                perror("pthread_mutex_unlock failed");
                exit(EXIT_FAILURE);
            }
        }
    }
    fprintf(stderr, "Done decreasing variable.\n");

    return NULL;
}

```

```

int main(int argc, char *argv[])
{
    int val, ret, ok;
    pthread_t t1, t2;

    val = 0;

    ret = pthread_create(&t1, NULL, increase_fn, &val);
    if (ret) {
        perror_thread(ret, "pthread_create");
        exit(1);
    }
    ret = pthread_create(&t2, NULL, decrease_fn, &val);
    if (ret) {
        perror_thread(ret, "pthread_create");
        exit(1);
    }

    /*
     * Wait for threads to terminate
     */
    ret = pthread_join(t1, NULL);
    if (ret)
        perror_thread(ret, "pthread_join");
    ret = pthread_join(t2, NULL);
    if (ret)
        perror_thread(ret, "pthread_join");

    ok = (val == 0);

    printf("%sOK, val = %d.\n", ok ? "" : "NOT ", val);

    return ok;
}
~

```

Έξοδος:

Με atomic

```

About to increase variable 10000000 times
About to decrease variable 10000000 times
Done decreasing variable.
Done increasing variable.
OK, val = 0.

```

Με mutex

```
About to increase variable 10000000 times  
About to decrease variable 10000000 times  
Done increasing variable.  
Done decreasing variable.  
OK, val = 0.
```

Ερώτηση 1

Είναι πιο γρήγορο το πρόγραμμα χωρίς συγχρονισμό.
Αυτό είναι λογικό καθώς στο ασύγχρονο μπορούν να μπουν στο critical ταυτόχρονα.

Συγχρονισμένα:

Atomic:

```
real    0m0.484s  
user    0m0.952s  
sys     0m0.000s
```

Mutex:

```
real    0m26.922s  
user    0m26.588s  
sys     0m27.184s
```

Ασύγχρονα:

Atomic:

```
real    0m0.056s  
user    0m0.076s  
sys     0m0.000s
```

Mutex:

```
real    0m0.055s  
user    0m0.072s  
sys     0m0.000s
```

Ερώτηση 2

Οι ατομικές εντολές είναι πιο γρήγορες (έχουμε ενεργό αναμονή). Αυτό είναι λογικό καθώς το κρίσιμο τμήμα διαρκεί λίγο οπότε θα ήταν πιο χρονοβόρο να εμπλακεί το ΛΣ ώστε να βάλει τις διεργασίες σε λίστα αναμονής και μετά να τις ξυπνήσει. Σε περίπτωση που το κρίσιμο τμήμα ήταν εκτενές θα ήταν γρηγορότερα τα mutexes καθώς θα αποδεσμεύονταν πόροι της ΚΜΕ που θα αξιοποιούνται για άλλες διεργασίες.

Ερώτηση 3

```
add_and_fetch
.loc 1 55 0
    movl    $1, %eax
    lock xaddl    %eax, ptr(%rip)
    addl    $1, %eax
    movl    %eax, x(%rip)
```

```
sub_and_fetch
    movl    $-1, %eax
    negl    %eax
    movl    %eax, %edx
    movl    %edx, %eax
    lock xaddl    %eax, ptr(%rip)
    addl    %edx, %eax
    movl    %eax, x(%rip)
```

Ερώτηση 4

```
call    __errno_location
    movq    %rax, %rbx
    movl    $mutex, %edi
    call    pthread_mutex_lock
    movl    %eax, (%rbx)
    movl    (%rbx), %eax
    testl    %eax, %eax
    je      .L3
```

```
.L3:
    .loc 1 69 0
    movq    -32(%rbp), %rax
    movl    (%rax), %eax
    leal    1(%rax), %edx
    movq    -32(%rbp), %rax
    movl    %edx, (%rax)
```

Άσκηση 2

Στον κώδικα που μας δόθηκε τροποποιήσαμε μόνο τις συναρτήσεις main και *compute and output_mandel_line

Κώδικας:

```
int main(int argc, char *argv[])
{
    //int n = argv[1];
    n = atoi(argv[1]);
    pthread_t t[n];

    sems = (sem_t *) malloc(sizeof(sem_t)*(n));

    // sem_init(&seml, 0, 0); // initialize sem1 to 0
    // sem_init(&sem2, 0, 1); // initialize sem2 to 1

    int lines[n], i, ret;

    xstep = (xmax - xmin) / x_chars;
    ystep = (ymax - ymin) / y_chars;

    for(i = 0; i < n-1; i++) {

        //printf("%d \n", n);
        sem_init(&sems[i], 0, 0);
    }
    sem_init(&sems[n-1], 0, 1);

    /*
     * draw the Mandelbrot Set, one line at a time.
     * Output is sent to file descriptor '1', i.e., standard output.
     */

    for(i = 0; i < n; i++){

        lines[i] = i;
        //printf("%d \n", line);
        ret = pthread_create(&t[i], NULL, compute_and_output_mandel_line, &lines[i]);
        if (ret) {
            perror_thread(ret, "pthread_create");
            exit(1);
        }

    }

    reset_xterm_color(1);

    for(i = 0; i < n; i++) pthread_join(t[i], NULL);

    /*
     *
     */
    reset_xterm_color(1);

    return 0;
}
```

```

int n;
sem_t *sems;

void *compute_and_output_mandel_line(void *arg)
{
    /*
     * A temporary array, used to hold color values for the line being drawn
     */

    volatile int *help = arg; /*
                               * volatile int* p; is a pointer to an int that the compiler will treat as volatile.
                               * This means that the compiler will assume that it is possible for the variable that p
                               * is pointing at to have changed even if there is nothing in the source code to suggest
                               * that this might occur.
                               * The only effect of volatile is to warn the compiler that the value of x might be changed
                               * from another thread.
                               */

    int line = *help;
    int color_val[x_chars];
    int j;

```

```

    int line = *help;
    int color_val[x_chars];
    int j;

    for(j = line; j < y_chars; j = j + n){

        signal(SIGINT, sigintHandler);

        compute_mandel_line(j, color_val);

        int mod = j % n;          // shows the thread that is currently running

        if(mod == 0) {

            sem_wait(&sems[n-1]);
            output_mandel_line(1, color_val);
            sem_post(&sems[0]);

        }
        /*

        else if(mod == n-1){

            sem_wait(&sems[n-2]);
            output_mandel_line(1, color_val);
            sem_post(&sems[n-1]);

        }
        */

        else{

            sem_wait(&sems[mod-1]);
            output_mandel_line(1, color_val);
            sem_post(&sems[mod]);

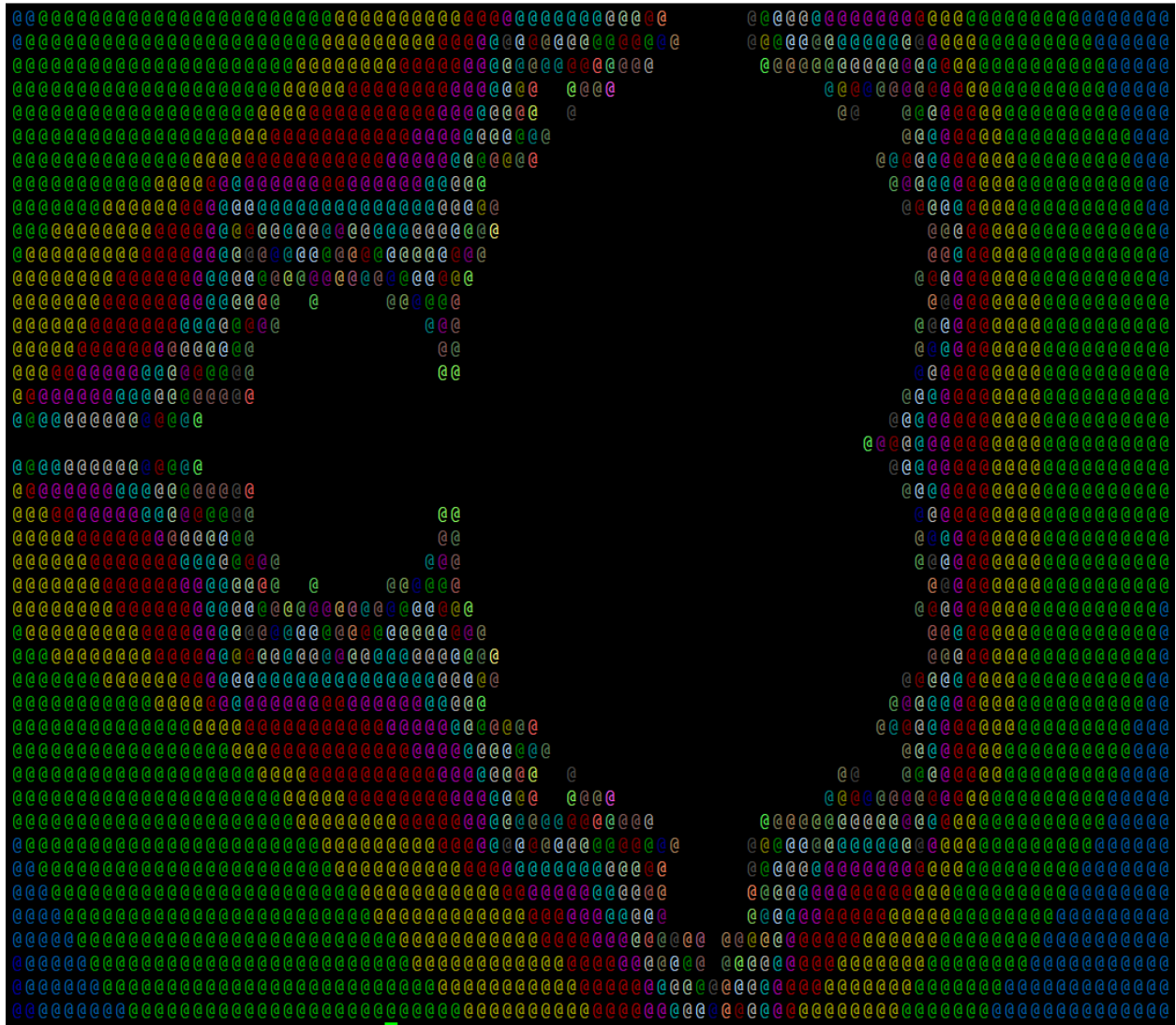
        }

    }
}

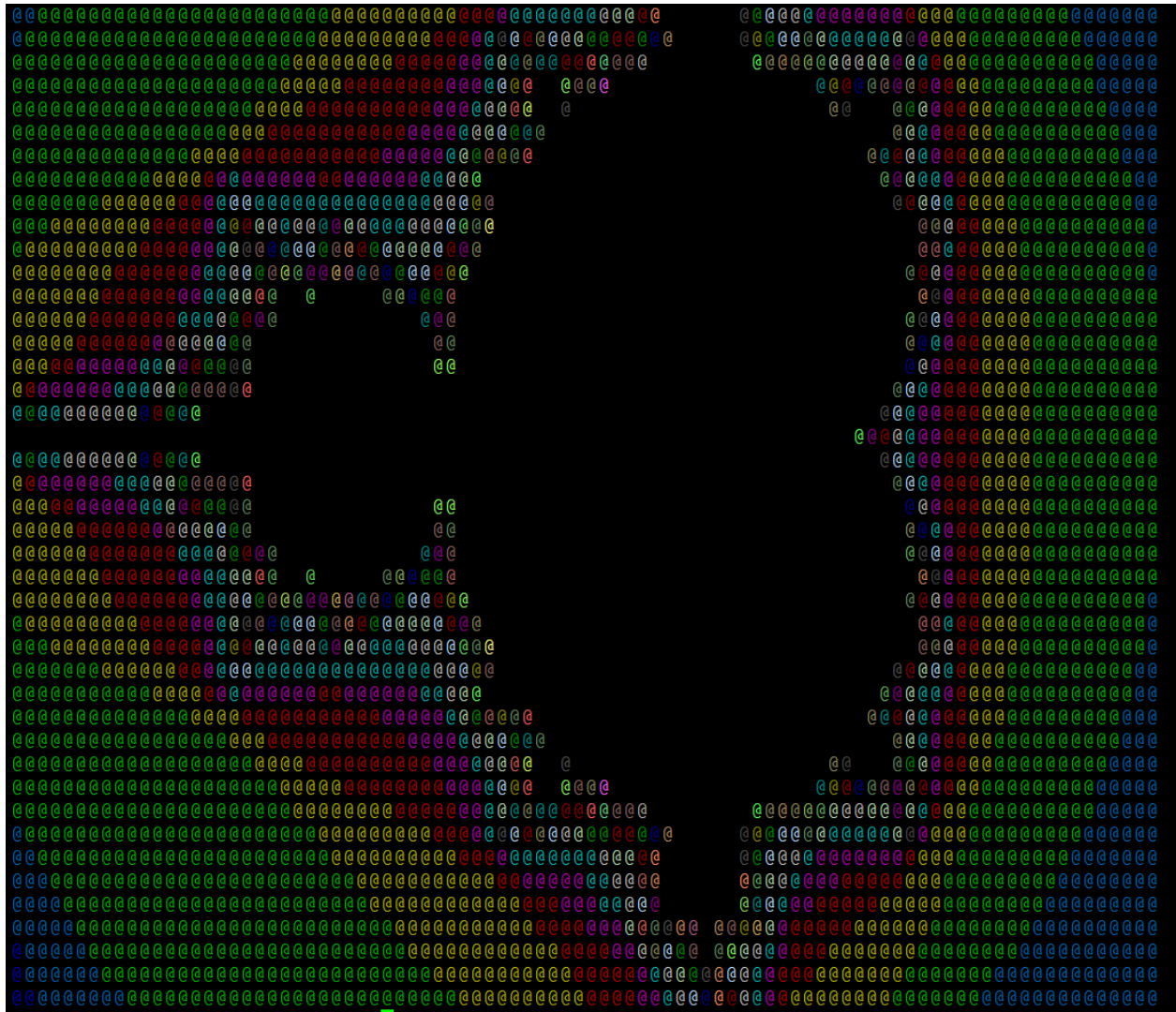
```

Έξοδος:

Με 2 νήματα



Με 5 νήματα



Ερώτηση 1

Χρειάζονται η σημαφόροι

Ερώτηση 2

Παράλληλο: όσα περισσότερα βάζουμε μέχρι το όριο των επεξεργαστών που έχει ο υπολογιστής μας, τόσο πιο γρήγορα έχουμε αποτέλεσμα

```
real 0m0.526s
user 0m0.996s
sys 0m0.016s
```

Σειριακό:

```
real 0m1.028s
user 0m0.988s
sys 0m0.020s
```

Προφανώς το παράλληλο είναι πιο γρήγορο.

Ερώτηση 3

Εμφανίζει επιτάχυνση

Χρόνος εκτέλεσης με 1 νήμα

```
real 0m1.033s
user 0m0.988s
sys 0m0.020s
```

Χρόνος εκτέλεσης με 2 νήματα

```
real 0m0.525s
user 0m0.996s
sys 0m0.016s
```

Χρόνος εκτέλεσης με 8 νήματα

```
real 0m0.171s
user 0m0.980s
sys 0m0.040s
```

Τον μικρότερο χρόνο εκτέλεσης τον πετυχαίνουμε όταν βάζουμε τόσα νήματα , όσοι είναι και οι επεξεργαστές στον υπολογιστή που χρησιμοποιούμε. Εάν βάλουμε μικρότερο αριθμό νημάτων τότε δεν χρησιμοποιούμε όλους τους δυνατούς πόρους, ενώ αν βάλουμε περισσότερα δεν υπάρχουν αρκετοί πόροι για να μας εξυπηρετήσουν. Άρα, έχουμε μέγιστο χρόνο για πλήθος νημάτων = πλήθος επεξεργαστών.

Χρειάζεται μόνο την φάση εξόδου όπως και έχουμε βάλει.

Ερώτηση 4

Έχουμε βάλει signal handler και όταν γίνει το CTR+C το πρόγραμμα δεν κάνει τίποτα και συνεχίζει από κει που είχε μείνει.

Άσκηση 3

Κώδικας:

```
void child_enter(struct thread_info_struct *thr)
{
    if (!thr->is_child) {
        fprintf(stderr, "Internal error: %s called for a Teacher thread.\n",
            __func__);
        exit(1);
    }

    fprintf(stderr, "THREAD %d: CHILD ENTER\n", thr->thrid);

    pthread_mutex_lock(&thr->kg->mutex);

    while (1){
        int children;
        children = thr->kg->vc;

        int teachers;
        teachers = thr->kg->vt;

        int ratio;
        ratio = thr->kg->ratio;

        if ((teachers)*(ratio) < (children + 1))
            pthread_cond_wait(&thr->kg->cond1, &thr->kg->mutex);

        else{
            ++(thr->kg->vc);
            break;
        }
    }

    pthread_mutex_unlock(&thr->kg->mutex);
}
```

```

void child_exit(struct thread_info_struct *thr)
{
    if (!thr->is_child) {
        fprintf(stderr, "Internal error: %s called for a Teacher thread.\n",
                __func__);
        exit(1);
    }

    fprintf(stderr, "THREAD %d: CHILD EXIT\n", thr->thrid);

    pthread_mutex_lock(&thr->kg->mutex);
    --(thr->kg->vc);

    int children;
    children = thr->kg->vc;

    int teachers;
    teachers = thr->kg->vt;

    int ratio;
    ratio = thr->kg->ratio;

    if (((teachers-1)*(ratio)) >= (children))
        pthread_cond_signal(&thr->kg->cond2);

    pthread_mutex_unlock(&thr->kg->mutex);
}

```

```

void teacher_enter(struct thread_info_struct *thr)
{
    if (thr->is_child) {
        fprintf(stderr, "Internal error: %s called for a Child thread.\n",
                __func__);
        exit(1);
    }

    fprintf(stderr, "THREAD %d: TEACHER ENTER\n", thr->thrid);

    pthread_mutex_lock(&thr->kg->mutex);
    ++(thr->kg->vt);

    pthread_cond_broadcast(&thr->kg->cond1);

    pthread_mutex_unlock(&thr->kg->mutex);
}

```

```

void teacher_exit(struct thread_info_struct *thr)
{
    if (thr->is_child) {
        fprintf(stderr, "Internal error: %s called for a Child thread.\n",
            __func__);
        exit(1);
    }

    fprintf(stderr, "THREAD %d: TEACHER EXIT\n", thr->thrid);

    pthread_mutex_lock(&thr->kg->mutex);
    int children;
    children = thr->kg->vc;

    int teachers;
    teachers = thr->kg->vt;

    int ratio;
    ratio = thr->kg->ratio;

    if (((teachers - 1) * (ratio)) < (children))
        pthread_cond_wait(&thr->kg->cond2, &thr->kg->mutex);
    --(thr->kg->vt);
    pthread_mutex_unlock(&thr->kg->mutex);
}

```

Ερώτηση 1

Γενικά , όταν ένας καθηγητής θέλει να βγει από το σχολείο και δεν μπορεί μπαίνει σε wait και όταν του δοθεί σήμα από την child_exit βγαίνει.

Όταν ένα παιδί θέλει να μπει και δεν μπορεί, κάνει wait και έπειτα όταν του δοθεί σήμα από την teacher_enter μπαίνει.

Για το αν θα μπουν παιδιά ή θα βγει καθηγητής εξαρτάται από το αν θα μπει καθηγητής ή αν θα βγει επαρκής αριθμός παιδιών αντίστοιχα.

Πιο συγκεκριμένα, έχουμε τις εξής περιπτώσεις:

Περίπτωση 1η

Ο καθηγητής περιμένει να βγει και παράλληλα βγαίνουν κάποια παδιά, το πλήθος των οποίων δεν επιτρέπει την έξοδο του καθηγητή. Στη συνέχεια θα μπορέσουν να μπουν κάποια παιδιά από αυτά που καταφτάνουν και ο καθηγητής θα παραμείνει εντός του νηπιαγωγείου.

Περίπτωση 2η

Βγαίνει επαρκής αριθμός παιδιών και ο καθηγητής καταφέρνει να βγει. Ο αριθμός των νέων παιδιών που θα μπουν εξαρτάται από το πόσο μεγαλύτερο είναι το πλήθος των παιδιών που βγαίνουν σε σχέση με τον αναγκαίο αριθμό για την έξοδο του καθηγητή.

Περίπτωση 3η

Υπάρχει χώρος για είσοδο νέων παιδιών και στη συνέχεια βγαίνουν παιδιά, το πλήθος των οποίων δεν επιτρέπει την έξοδο του καθηγητή.

Περίπτωση 4η

Υπάρχει χώρος για είσοδο νέων παιδιών και στη συνέχεια βγαίνουν παιδιά, το πλήθος των οποίων επιτρέπει την έξοδο του καθηγητή.

Να σημειωθεί ότι οι παραπάνω περιπτώσεις αποτελούν μία απλούστευση καθώς δεν είναι αναγκαίο να βγαίνουν όσα παιδιά περιμένουν όλα μαζί (ή αντίστοιχα να μπαίνουν όλα μαζί). Μπορεί ένα παιδί να μπαίνει, στη συνέχεια δύο να βγαίνουν και ο καθηγητής να μην καταφέρει να βγει ποτέ

Ερώτηση 2

Η κατάσταση που περιγράψαμε στην ερώτηση 1 είναι μία κατάσταση συναγωνισμού. Για παράδειγμα στην 2η περίπτωση υπάρχει κατάσταση συναγωνισμού μεταξύ των παιδιών που θέλουν να μπουν καθώς μπορεί ο αριθμός των καθηγητών που έμειναν (αφού βγήκε ο προηγούμενος) να μην επαρκεί για είσοδο όλων των παιδιών που περιμένουν.